

Budget-Constrained Non-Stationary LLM Routing via Primal-Dual Contextual Bandits

Abstract

Large Language Model (LLM) routing dynamically directs user queries across a portfolio of models—from lightweight open weights to frontier APIs—optimizing the trade-off between inference cost and response quality. While static classifiers are commonly deployed for this task, production environments exhibit significant non-stationarity: models receive silent performance updates, inference providers drop prices abruptly, and aggregate API limits impose strict dollar budgets. In this paper, we present an open-source framework for non-stationary LLM routing via cost-aware contextual bandits. We demonstrate three key capabilities critical for production deployments: (1) an online Lagrangian relaxation (Primal-Dual Contextual Bandits with Knapsacks) that enforces strict average-cost budgets dynamically, eliminating the need for offline hyperparameter tuning of cost penalties; (2) a geometric forgetting mechanism that enables the router to adapt gracefully to model quality shifts without suffering catastrophic forgetting of its initial offline-trained priors; and (3) the application of a rigorous hyperparameter tuning methodology to LLM routing that jointly optimizes exploration, prior strength, and forgetting factors. While established in other domains, adapting this tuning specifically addresses the uniquely contending requirements of LLM routing, allowing the system to strike an optimal balance between stable stationary exploitation and rapid agility under distribution shifts.

Through empirical evaluation on an established dataset of 1,785 realistic prompts, spanning a portfolio with a 500× cost differential (Llama-3.1-8B to Gemini-2.5-Pro), we demonstrate our system’s robustness. Under simulated reward drift, our approach significantly outperforms both frozen static policies and naive infinite-memory bandits. Critically, we demonstrate that when cost drift (e.g., a mid-cycle price drop) and budget constraints interact, our adaptive budget pacer automatically exploits the freed capital to route towards higher-quality frontier models, delivering up to +6.7% quality lift while holding spend within 2% of the dollar target. Static penalty approaches, by contrast, deviate from the budget by up to 162%, translating to either wasted quality headroom or SLA-violating cost overruns. We further show that the bandit’s exploration mechanism enables *cold-start model onboarding*: a single API call adds a previously unseen model with no offline evaluation or warmup priors, and the router discovers its quality–cost niche from live traffic within hundreds of requests while maintaining budget compliance—eliminating the days-to-weeks recalibration cycle required by static routers. We release the fully documented routing framework to

the open-source community to facilitate both further research and immediate production use.

ACM Reference Format:

. 2026. Budget-Constrained Non-Stationary LLM Routing via Primal-Dual Contextual Bandits. In . ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/1122445.1122456>

1 Introduction

Large language model (LLM) inference increasingly draws from multi-provider portfolios [3], spanning lightweight open-weights models to frontier APIs. Because the optimal model varies across tasks, domains, and individual prompts, **LLM routing**—dynamically selecting the best model per request—has emerged as a critical lever for managing cost and quality at scale [5, 6, 11, 12]. In production, an engineering team may route millions of requests daily across models spanning a 500× price range (e.g., \$0.00003/req for Llama-3.1-8B vs. \$0.015/req for Gemini-2.5-Pro) and must keep average spend within a strict API budget.

Crucially, real enterprise portfolios are not binary “strong-vs.-weak” pairs. Organizations typically maintain three or more model tiers—budget, mid-range, and frontier—and often deploy multiple models per tier for redundancy, latency, or specialization (e.g., a code-focused model alongside a general-purpose model at the same price point). The number of viable options continues to grow: major providers now offer 5–15 distinct model endpoints each, and multi-provider deployments routinely span $K=5$ –10 or more candidates. The value of a larger portfolio is not that more models necessarily produce a better quality frontier—a single frontier API may dominate on raw quality—but rather that more models at distinct price points give the router *more degrees of freedom to find quality within a budget*. When budgets are tight, a $K=2$ router can only choose “cheap” or “expensive”; with $K\geq 3$ it can discover that a mid-tier model matches frontier quality on easy prompts at a fraction of the cost, stretching the budget to afford frontier calls where they matter most. The same logic applies to non-stationarity: when a model degrades, a richer portfolio increases the chance that an alternative maintains quality at a comparable cost; when a provider cuts prices, more models mean more opportunities for the router to reallocate freed budget toward quality improvements.

Many existing routers operate as static classifiers trained offline on evaluation data [5, 6, 11]. While highly effective in stationary environments, frozen policies risk degradation when deployed in real-world scenarios due to three critical factors. First, **deployment distributions drift**: prompt mixes vary by use case and drift over time from offline training data [2]. Second, **model quality is non-stationary**: providers silently update APIs in ways that can subtly shift downstream performance on specific tasks [4, 10], meaning a router trained on historical data inherits a *stale policy*. Third, **model costs drop frequently**: rapid frontier releases often trigger price cuts [14]. If a static router maintains a fixed routing distribution during a price drop, it merely pockets the savings, leaving potential

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, Washington, DC, USA

© 2026 ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/1122445.1122456>

quality gains—such as upgrading a larger percentage of traffic to the now-cheaper frontier model—unexploited.

To address the need for online adaptation, recent work has formulated routing as a contextual bandit or reinforcement learning problem [9, 12, 13, 15]. These approaches learn dynamically from feedback (e.g., user ratings or automated judge scores) to adjust their policies over time. However, three critical challenges remain for deploying these adaptive systems in budget-constrained enterprise environments: 1. **Budget Enforcement**: Existing cost-aware bandits typically scalarize cost into the reward via a static penalty weight λ [11, 15]. This requires offline tuning to translate an abstract weight into a dollar outcome, and must be manually re-tuned whenever pricing changes or the traffic distribution shifts. 2. **Catastrophic Memory vs. Agility**: Standard algorithms like LinUCB often use infinite memory (forgetting factor $\gamma = 1.0$). While optimal for stationary settings, this causes the bandit to react far too slowly when a previously high-performing model suddenly degrades in quality, accumulating massive regret during the transition. 3. **Contending Contribution Requirements**: Tuning an adaptive system to simultaneously maintain stable stationary performance (which favors infinite memory and low exploration) and agile non-stationary adaptation (which favors aggressive forgetting and high exploration) creates conflicting optimization objectives. Standard cross-validation optimizes solely for stationary metrics, leading to brittle real-world deployments.

Left unaddressed, these gaps carry direct financial consequences: a static penalty that drifts from its budget target either wastes quality headroom (under-spend) or triggers cost overruns (over-spend), and a router that cannot detect a price drop leaves freed budget idle rather than reinvesting it in application quality.

We address these practical deployment challenges with **BanditGPT**, an open-source contextual bandit framework explicitly designed for budget-constrained, non-stationary LLM routing. We make four key contributions, evaluated on a dataset of 1,785 prompts scored by a continuous three-factor rubric (cross-judge robustness validated in Appendix ??) across a canonical three-tier model portfolio:

- (1) **Direct Budget Pacing via Primal-Dual CBwK**: We formulate routing as a Contextual Bandit with Knapsacks (CBwK) and implement an online Lagrangian relaxation (BudgetPacer) that accepts a direct dollar-budget target. By applying exponential moving average (EMA) smoothing to handle the extreme variance in LLM pricing, our system dynamically adjusts penalties to achieve exact budget compliance while automatically tracing the Pareto frontier, eliminating offline λ tuning.
- (2) **Robust Adaptation via Geometric Forgetting**: To combat silent model updates (reward drift), we integrate a geometric forgetting mechanism ($\gamma = 0.995$) coupled with offline-to-online warmup priors. This enables the router to adapt rapidly to quality shifts without suffering catastrophic forgetting of the initial prompt-preference mapping, drastically reducing cumulative regret under drift compared to naive bandits ($\gamma = 1.0$) and frozen policies.
- (3) **Exploiting Cost Drift under Budget Constraints**: We demonstrate that when a model provider drops its price mid-cycle, the BudgetPacer automatically detects and exploits the opportunity. Unlike static baselines that fail to adapt, the primal-dual

mechanism produces qualitatively distinct responses across budget regimes: when the constraint becomes slack, λ_t decays to zero and the router reverts to quality-only optimization; when the constraint remains binding, λ_t rises and concentrates traffic on the cost-efficient frontier. In both cases, BanditGPT maintains precise budget compliance ($0.98\text{--}1.02\times$ target) while delivering significant reward lifts ($+0.017$ to $+0.065$).

- (4) **Rigorous Hyperparameter Optimization for Shift**: We adapt joint hyperparameter tuning (exploration α , prior strength n_{eff} , forgetting factor γ) to the LLM routing domain. While utilized in other fields, this approach directly resolves the uniquely contending requirements of our system, structurally balancing stable stationary exploitation with rapid agility under distribution shifts where standard cross-validation fails.

Beyond these four contributions, we demonstrate that BanditGPT’s exploration mechanism enables *cold-start model onboarding*: a single `register_model()` call integrates a previously unseen model—with no offline evaluation or warmup priors—into a budget-constrained router, and the bandit discovers its quality–cost niche from live traffic within hundreds of requests (Section ??; Appendix ??).

By releasing the framework open-source, we aim to provide an immediate pathway for practitioners managing enterprise LLM gateways to replace manual recalibration with autonomous Primal-Dual pacing, and to offer the research community a reproducible testbed for non-stationary routing. The remainder of this paper is organized as follows: Section 2 surveys related work; Section 3 formulates the budget-constrained non-stationary routing problem; Section 4 presents the system architecture; Section 5 details the experimental setup; Section 6 discusses the empirical evaluation; and Section 7 outlines practical deployment considerations.

2 Related Work

LLM routing has evolved rapidly from cascading heuristics to supervised classifiers, and more recently, to adaptive routing systems. In this section, we situate BanditGPT within the landscape of these approaches, highlighting the gaps in addressing production non-stationarity and dynamic budget constraints.

2.1 Static and Supervised Routers

Early approaches to cost-aware LLM routing relied on static rules or cascading heuristics (e.g., FrugalGPT [3]), where a system queries models sequentially from cheapest to most expensive, stopping when a predicted quality threshold is met. While simple to implement, cascades introduce highly variable latency, as complex queries must await sequential failures before reaching a capable model.

More recent systems formulate routing as a supervised classification task trained offline on preference or evaluation data. RouteLLM [11] uses matrix factorization on pairwise preference data to map queries to models via a cost threshold. GraphRouter [6] constructs a heterogeneous graph over tasks, queries, and LLMs, using edge prediction to estimate per-model quality and cost, allowing it to generalize to newly added LLMs without full retraining. RouterDC [5] learns joint embeddings of queries and LLMs via dual

contrastive losses, routing to the model with the highest similarity score.

While supervised routers achieve strong performance when deployed on stationary distributions that match their training data, they exhibit two critical flaws in production environments. First, they deploy *frozen policies* that cannot adapt to non-stationarity (e.g., silent model updates or shifting user prompts). Second, they enforce budget constraints via fixed, offline-tuned thresholds or cost penalties. If a model provider unexpectedly drops prices, these static systems fail to dynamically exploit the new cost-quality trade-off, either under-spending their budget or requiring a manual, offline retraining and recalibration cycle.

2.2 Adaptive and Bandit-Based Routers

To enable online adaptation, a recent wave of research has formulated LLM routing as a contextual bandit or reinforcement learning (RL) problem, allowing systems to learn continuously from deployment feedback.

PILOT [12] extends LinUCB [8] with *preference-prior informed* initialization, injecting LLM embeddings learned offline from human preference data to mitigate the cold-start problem. To manage costs, PILOT employs an online multi-choice knapsack algorithm to optimize cumulative cost over a known, finite horizon. While theoretically grounded, the knapsack approach requires knowing the total number of queries in advance, making it difficult to apply to open-ended, continuous production streams where request volume fluctuates.

BaRP [15] trains a contextual bandit using entropy-regularized policy gradients, conditioning routing on a user preference vector to allow inference-time adjustments to the cost-quality trade-off. LLM Bandit [9] utilizes a multi-armed bandit with preference-conditioned routing, incorporating model identity vectors to generalize to unseen models. Similarly, xRouter [13] takes an RL approach, fine-tuning a small language model to act as both a router and a responder, trained end-to-end with a cost-aware reward signal. Finally, recent work on contextual queueing bandits (e.g., ACQB [?]) leverages implicit feedback from user retries to jointly optimize routing and scheduling in conversational services.

2.3 Positioning BanditGPT

BanditGPT builds upon the foundation of contextual bandits but introduces critical structural advances designed specifically for the realities of open-ended, budget-constrained production environments:

- (1) **Continuous Budget Pacing:** Unlike PILOT’s finite-horizon knapsack or the static scalarized cost penalties used in RouteLLM and BaRP, BanditGPT utilizes a Primal-Dual Contextual Bandit with Knapsacks (CBwK) framework. This *BudgetPacer* strictly enforces a per-request average cost target (e.g., “\$0.005 per request”) over an infinite horizon, automatically adjusting to price drops or traffic shifts without offline recalibration.
- (2) **Explicit Handling of Non-Stationarity:** While existing bandit routers continuously learn, they often employ infinite memory (e.g., standard LinUCB). When model quality degrades (reward drift), infinite memory causes catastrophic inertia, as the

router slowly unlearns thousands of stale historical observations. BanditGPT introduces a principled geometric forgetting mechanism that dynamically discounts stale evidence, enabling rapid adaptation to quality shifts while preserving enough historical context to avoid volatile, uninformed routing.

- (3) **Production State Management:** Existing bandit routers maintain learned state exclusively in process memory, making them unable to recover from restarts or incorporate delayed human feedback (RLHF). BanditGPT provides pluggable context persistence (in-memory or SQLite-backed) for asynchronous feedback ingestion and atomic state checkpointing for crash recovery, bridging the gap between research prototypes and deployable systems (Section 7).
- (4) **Native $K \geq 3$ Multi-Model Support:** Most existing routers—both supervised (RouteLLM [11]) and adaptive (BaRP [15])—are designed around a binary strong/weak decision, using pairwise preference data or binary cost thresholds to triage between exactly two models. This $K=2$ framing limits a router’s ability to operate under budget constraints and non-stationarity. With only two arms, the router has a single lever—what fraction of traffic goes to the expensive model—leaving no room to discover that a mid-tier alternative delivers comparable quality at a fraction of the cost. When rewards shift or prices drop, a richer portfolio gives the router more options to reallocate traffic and maintain quality within budget; with $K=2$, every such event collapses to the same binary trade-off. BanditGPT’s contextual bandit formulation is natively K -armed: each model is an independent arm with its own learned reward surface, the BudgetPacer’s Lagrangian penalty operates over the full cost spectrum, and new arms are added at runtime via a single API call without architectural changes (Appendix ??).

3 Problem Formulation

We formalize LLM routing as an online decision-making problem under budget constraints, where the environment is non-stationary with respect to both model quality (reward drift) and model pricing (cost drift).

3.1 Contextual Bandit Routing

Let $t \in \{1, 2, \dots, T\}$ denote a discrete sequence of user requests (prompts). At each step t , the router receives a context vector $x_t \in \mathbb{R}^d$ representing the prompt, derived from a pre-trained embedding model (e.g., SentenceTransformers). The router must select a single language model a_t from a discrete portfolio of available models $\mathcal{A} = \{1, \dots, K\}$.

Upon executing the prompt on model a_t , the system observes a scalar reward $r_t \in [0, 1]$ indicating the quality of the response. This reward may be derived from an automated LLM-as-a-judge, explicit user feedback (e.g., thumbs up/down), or an implicit task completion metric. Crucially, the router operates under *bandit feedback*: it only observes the reward for the chosen model a_t and does not know what the counterfactual rewards would have been for the unselected models.

The primary objective is to learn a routing policy π that maximizes the expected cumulative reward:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^T r_{t, \pi(x_t)} \right] \quad (1)$$

3.2 The Budget Constraint

In production, each model $a \in \mathcal{A}$ incurs a known, deterministic inference cost $c_a(x_t)$ (typically priced per input and output token). Because the models in a portfolio often span orders of magnitude in price—from open-weights models costing fractions of a cent to frontier APIs costing several cents per request—the router must strictly manage aggregate spend.

We formulate this as a *Contextual Bandit with Knapsacks* (CBwK) [1] problem. The system is given a target average budget B (e.g., “average \$0.005 per req”). The policy π must maximize cumulative reward subject to the constraint that the long-run average cost does not exceed B :

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T c_{\pi(x_t)}(x_t) \leq B \quad (2)$$

The standard approach to this constraint in the routing literature is to scalarize the reward using a fixed penalty weight $\lambda > 0$, optimizing a penalized utility $u_{a,t} = r_{a,t} - \lambda c_a(x_t)$ [11]. However, selecting λ requires extensive offline simulation on historical data to find the weight that happens to yield an average cost of B . If the distribution of prompts shifts, or if the portfolio changes, the realized cost will drift away from B , requiring manual recalibration.

3.3 Non-Stationarity

Real-world LLM deployments are fundamentally non-stationary. We define two specific types of environment drift that a production router must handle autonomously:

Reward Drift (Quality Shifts). LLM providers silently update their models behind APIs, or internal teams push new versions of fine-tuned open-weight models. Formally, the conditional expected reward function $f_a(x) = \mathbb{E}[r_t | x, a]$ changes at some unknown time τ , such that $f_a^{(t < \tau)}(x) \neq f_a^{(t \geq \tau)}(x)$. A router must detect this shift and rapidly unlearn its historical estimates for model a , avoiding a stale policy that continues routing traffic to a degraded model.

Cost Drift (Price Drops). API providers frequently reduce inference prices to remain competitive [14]. Formally, the cost function $c_a(x)$ decreases at time τ . If the router uses a fixed, offline-tuned penalty λ , it will continue to apply the same relative penalty to model a . While this mechanically reduces the total spend (pocketing the savings), it fails to recognize that model a is now a relatively better value. An optimal router should dynamically exploit the newly freed budget to route more challenging queries to the newly-cheaper, higher-quality model, maximizing overall application quality while remaining precisely at the budget limit B .

4 Methodology: The BanditGPT System

We present the architecture of BanditGPT, an open-source contextual bandit routing framework. This section details the end-to-end routing pipeline, our Primal-Dual budget pacing mechanism,

and the geometric forgetting approach for non-stationary environments.

4.1 System Architecture Overview

BanditGPT operates as an asynchronous, closed-loop routing system (Figure 1). At inference time (step t), a user query x_t is processed by a lightweight feature extractor (e.g., a SentenceTransformer embedding model), which passes the query vector to the BanditRouter. The router maintains a set of sufficient statistics for each model $a \in \mathcal{A}$, initialized via *offline-to-online warmup* priors to mitigate the cold-start problem. The router computes a score for each model, incorporating uncertainty (LinUCB) and subtracting a dynamic cost penalty provided by the BudgetPacer. The router then selects the model a_t with the highest score, returning its response to the user.

After the query completes and is evaluated (either synchronously by an automated judge, or asynchronously via delayed user feedback), the reward r_t and exact cost c_t are fed back into the system. The BanditRouter updates its reward estimates (incorporating geometric forgetting), while the BudgetPacer updates its internal dual variable λ_t , adjusting the cost penalties for future requests to steer the system back toward the target budget B .

4.2 Primal-Dual Budget Pacing

To enforce strict, dynamically adjusting budget constraints without manual offline tuning of a static penalty weight λ_s , BanditGPT implements a Primal-Dual Contextual Bandit with Knapsacks (CBwK) formulation [1] via the BudgetPacer.

At each step t , the router selects model a_t by maximizing a budget-augmented utility function:

$$a_t = \arg \max_{a \in \mathcal{A}_t} \left[\underbrace{\hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}}_{\text{LinUCB score [8]}} - \underbrace{\lambda_s \tilde{c}_a}_{\text{static penalty}} - \underbrace{\lambda_t \tilde{c}_a}_{\text{dual penalty}} \right] \quad (3)$$

where $\mathcal{A}_t \subseteq \mathcal{A}$ is the set of models surviving a hard cost ceiling (detailed below), λ_t is a Lagrange multiplier updated after observing the realized cost c_t , and $\tilde{c}_a \in [0, 1]$ is the log-normalized unit cost defined as

$$\tilde{c}_a = \frac{\log c_a - \log c_{\min}}{\log c_{\max} - \log c_{\min}}, \quad (4)$$

with $c_{\min}$ and $c_{\max}$ fixed to the market-floor and market-ceiling blended rates of the model portfolio, respectively. Values at or below $c_{\min}$ map to 0; values at or above $c_{\max}$ map to 1. The logarithmic scale compresses the 500× cost differential into a bounded penalty that is commensurate with the $[0, 1]$ reward scale.

Dual-Variable Dynamics and EMA Smoothing. The core innovation of the BudgetPacer is how it tracks and updates the dual variable λ_t in response to the extreme variance in LLM pricing. Rather than updating λ_t directly from the raw per-request cost c_t , the pacer computes an *EMA-smoothed* cost signal:

$$\tilde{c}_t = (1 - \alpha_{\text{ema}}) \tilde{c}_{t-1} + \alpha_{\text{ema}} c_t \quad (5)$$

$$\lambda_{t+1} = \min \left(\tilde{\lambda}, \max(0, \lambda_t + \eta (\tilde{c}_t / B - 1)) \right) \quad (6)$$

Synchronous Inference Path

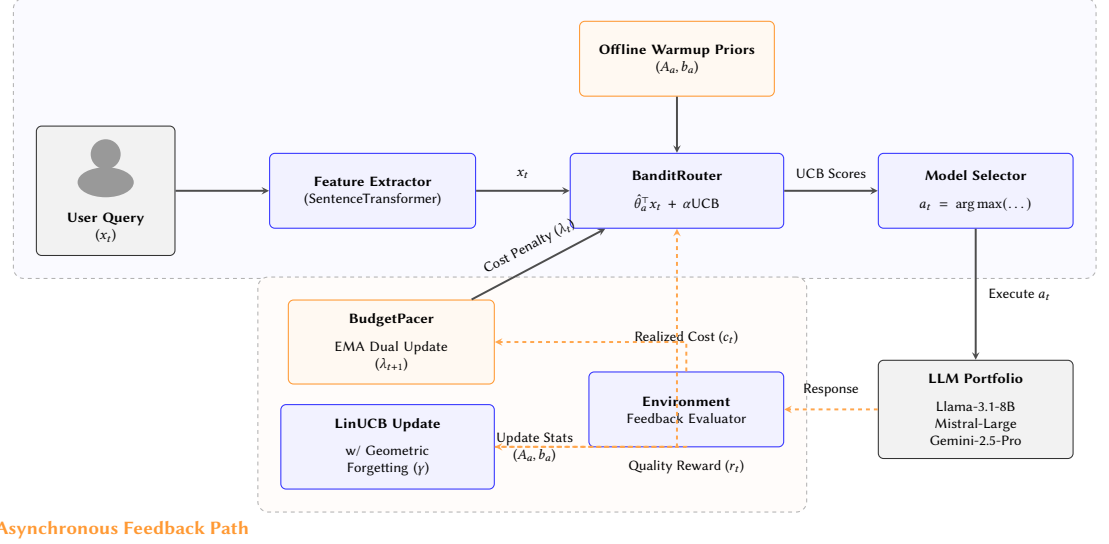


Figure 1: BanditGPT System Architecture. The system cleanly separates synchronous inference (blue) from asynchronous feedback (orange, color-blind friendly). During inference, the BudgetPacer dynamically applies a dual cost penalty (λ_t) to the LinUCB scores to enforce the budget constraint B . As feedback arrives asynchronously, the reward r_t updates the router’s statistics via geometric forgetting (γ), while the exact cost c_t updates the pacer’s dual variable (λ_{t+1}) via an EMA smoother.

where B is the user’s desired average cost per request, $\alpha_{\text{ema}} = 0.05$ is the smoothing factor, $\eta = 0.05$ is the step size, and $\bar{\lambda} = 5$ is a capacity cap.

Using the EMA $\bar{c}_t$ is critical. If a single \$0.015 Gemini request were routed, the raw ratio c_t/B could spike to $500\times$ the target B (e.g., if $B = \$0.00003$). This would produce massive, sawtooth oscillations in λ_t that never converge, rendering the penalty useless. The EMA smooths this variance, generating a stable gradient signal whose magnitude reflects the *average* cost regime over recent history.

Furthermore, normalizing the gradient by B (i.e., $\bar{c}_t/B$) makes the step size η portfolio-independent. The same η produces equivalent λ_t trajectories whether the cheapest model costs \$0.00003 or \$0.03, eliminating the need to retune hyperparameters when the pricing scale changes.

Two-Layer Enforcement (Adaptive Mode). The BudgetPacer combines two enforcement mechanisms simultaneously:

- **Hard ceiling (Safety Net):** When $\lambda_t > 0$, the candidate set $\mathcal{A}_t$ actively excludes models whose blended unit price exceeds the dynamic ceiling. This acts as a circuit breaker, preventing catastrophic overspend on any single request when the budget is severely constrained.
- **Soft penalty (Optimizer):** The dual penalty $\lambda_t \bar{c}_a$ biases the UCB score toward cheaper models proportionally to their cost. This allows fine-grained, context-dependent routing within the surviving candidate set, minimizing quality loss within the hard budget envelope.

4.3 Non-Stationary Adaptation via Geometric Forgetting

LLM routing in production is non-stationary: providers silently update models, creating reward drift. A naive LinUCB algorithm ($\gamma = 1.0$) accumulates infinite memory, severely slowing adaptation to a shifting reward landscape because it must overcome thousands of historical observations. Conversely, entirely discarding history discards the valuable prior knowledge required for stable, informed routing.

BanditGPT addresses non-stationarity through a *forgetting factor* $\gamma \in (0, 1]$ applied directly to the disjoint LinUCB sufficient statistics. At each step t , before incorporating the new observation (x_t, a_t, r_t) , the design matrix and reward accumulator for the chosen arm a_t are exponentially discounted:

$$A_{a_t} \leftarrow \gamma A_{a_t} + x_t x_t^\top \quad (7)$$

$$b_{a_t} \leftarrow \gamma b_{a_t} + r_t x_t \quad (8)$$

This exponential weighting [7] gives observations an effective half-life of $\ln 2 / (1 - \gamma)$ steps. We empirically find that $\gamma = 0.995$ strikes an optimal balance, creating an effective memory window of approximately 200 steps (see Section 6.4 for the formal selection procedure). Recent evidence dominates, allowing the router to adapt to shifts rapidly, while historical observations (including the critical offline warmup priors) are geometrically discounted but not erased entirely.

The parameter γ thus controls a bias-variance tradeoff between *stability* (trusting historical evidence to maintain high baseline quality) and *plasticity* (responding quickly to new evidence when models degrade).

4.4 Computational Efficiency

Because the router sits on the critical path of every LLM request, per-step latency must be negligible relative to model inference time. We ensure this through three design choices.

$O(d^2)$ updates via Sherman-Morrison. Naive LinUCB recomputes A_a^{-1} from scratch after each observation, costing $O(d^3)$. BanditGPT instead maintains a cached inverse A_a^{-1} and applies the Sherman-Morrison rank-1 update formula:

$$A_a^{-1} \leftarrow A_a^{-1} - \frac{(A_a^{-1}x_t)(x_t^\top A_a^{-1})}{1 + x_t^\top A_a^{-1}x_t}, \quad (9)$$

reducing each update to $O(d^2)$. For the production feature dimension $d=26$, this is $\sim 17,500\times$ faster than a full $O(d^3)$ inversion. A full $O(d^3)$ recomputation is triggered only as a rare numerical-stability safeguard when the regularization floor decays below 10% of its initial value under prolonged forgetting—in practice fewer than one in every thousand updates.

$O(Kd)$ arm selection. The expected reward $\hat{\theta}_a^\top x_t$ is a single $O(d)$ dot product per arm because $\hat{\theta}_a = A_a^{-1}b_a$ is cached and refreshed only at update time. The UCB exploration bonus requires one $O(d^2)$ quadratic form $x_t^\top A_a^{-1}x_t$ per arm. With $K=3$ arms and $d=26$, the full selection pass amounts to $\sim 2,100$ floating-point operations—well under a microsecond on commodity hardware.

Constant-time geometric decay. When $\gamma < 1$, discounting $A_a \leftarrow \gamma A_a$ requires multiplying the cached inverse by the reciprocal: $A_a^{-1} \leftarrow A_a^{-1}/\gamma$ —a scalar division that adds $O(d^2)$ element-wise operations but avoids a full matrix rebuild. Combined with the Sherman-Morrison correction for the new observation, the total per-update cost remains $O(d^2)$ regardless of whether forgetting is active.

Empirical validation. A latency microbenchmark (Appendix A.1) confirms that the production configuration ($d=26$, Sherman-Morrison) completes a full route + update cycle in $39\mu s$ (p50) and sustains $> 24,000$ req/s on a single CPU core, adding $< 0.004\%$ overhead to even the fastest LLM inference call. The combined effect of PCA dimensionality reduction and incremental inverse caching yields a $120\times$ throughput improvement over a naive full-inversion baseline operating on raw 384-D embeddings.

5 Experimental Setup

We evaluate BanditGPT on a canonical portfolio of three LLMs, spanning a large cost differential, and using realistic prompt data to simulate both stationary and non-stationary production environments. Table 1 summarises the data splits and model portfolio.

5.1 Datasets and Ground Truth

We collect 11,983 prompts from the LMSYS Chatbot Arena [16], reflecting real-world, open-ended user interactions spanning coding, reasoning, and conversational tasks. Each prompt is routed to all three models in the portfolio and evaluated by an automated LLM-as-a-judge.

Quality evaluation. We employ DeepSeek-R1 [?] as the evaluator via a structured chain-of-thought (CoT) judge prompt. The

prompt first elicits a concise reasoning analysis of the response, then requests scores on three continuous dimensions ($[0, 1]$): *Reasoning Quality*, *Instruction Following*, and *Communication Quality*. The composite scalar reward is the weighted average $r_t = 0.4 \cdot s_{\text{reason}} + 0.3 \cdot s_{\text{instr}} + 0.3 \cdot s_{\text{comm}} \in [0, 1]$. This continuous rubric provides the granular quality distinctions necessary for multi-arm routing; in the LLM routing setting, binary win/loss outcomes collapse into ties on a large fraction of prompts because adjacent-tier models produce comparable quality on most queries, rendering binary signals insufficient for distinguishing among $K > 2$ models (Section 7 provides a detailed analysis and supporting evidence from the literature). The full judge prompt is reproduced in Appendix ??.

For online learning, the bandit’s convergence rate scales directly with the reward gap between the optimal and suboptimal arms [?]; we therefore select the judge that maximises routing margins. DeepSeek-R1 produces the largest inter-model gaps and the highest fraction of actionable prompts among the judges we evaluated, while its oracle routing decisions capture $\geq 97\%$ of two independent supplementary judges’ oracle reward (Appendix ??). Recent multi-judge aggregation methods [?] target confounder-induced bias rather than the signal compression inherent in averaging judges with different scale ranges; applied to our panel, averaging reduces routing margins by $\sim 30\%$ (Appendix, Figure ??), making the single maximally discriminative judge the principled choice for bandit learning.

Data splits. The 11,983 prompts are partitioned into three disjoint sets via stratified sampling (by oracle difficulty, seed=42, 70/15/15 ratio). No prompt appears in more than one split:

- (1) **Train** ($n=8,374$): Used *offline* to build warmup priors that initialise the bandit’s sufficient statistics (A_a, b_a), addressing the cold-start problem. Never seen during online evaluation.
- (2) **Val** ($n=1,785$): Used for online learning and hyperparameter selection (Section 6.4) under stationary conditions.
- (3) **Test** ($n=1,824$): Evaluation split, held out from both warmup-prior fitting and hyperparameter selection. The bandit policy continues to receive feedback and adapt during the test trajectory, consistent with the standard cumulative-regret protocol for online learning [?]; “held-out” refers to data isolation, not policy freezing. Non-stationary scenarios (reward drift, cost drift) are constructed by manipulating reward or cost columns at a mid-trajectory phase boundary. All reported results use this split, averaged over multiple random seeds.

Context representation. Each prompt is encoded by the all-MiniLM-L6-v2 SentenceTransformer (384-D), L2-normalized, and projected to a 25-dimensional context vector via PCA, followed by a bias term ($d=26$). Dimensionality reduction is essential because LinUCB maintains a $d \times d$ design matrix A_a per arm, making per-step updates $O(d^2)$; projecting from 384 to 25 dimensions reduces this cost by $\sim 235\times$. We choose PCA over nonlinear alternatives (e.g., autoencoders) because LinUCB assumes a linear reward model $\mathbb{E}[r | x] = \theta^\top x$; PCA is the variance-maximizing linear projection, preserving the most signal under the same linearity assumption the bandit already makes. Although 25 components retain only $\sim 29\%$ of total embedding variance, the routing-relevant signal—which dimensions of prompt semantics predict *inter-model* reward gaps—concentrates in

Table 1: Dataset splits and model portfolio ($K=3$). All prompts are sourced from LMSYS Chatbot Arena and judged by DeepSeek-R1. The 500 $\times$ cost differential between the cheapest and most expensive model stresses the router’s quality–cost tradeoff.

Split	n	Role
Train	8,374	Warmup priors (offline)
Val	1,785	Online learning, hparam tuning
Test	1,824	Online evaluation (held-out data)
Total	11,983	

Model	Tier	Cost (\$/req)
Llama-3.1-8B	Budget	\$0.00003
Mistral-Large	Mid-tier	\$0.003
Gemini-2.5-Pro	Frontier	\$0.015

the leading principal components; a PCA-dimension ablation (Appendix ??) confirms the Pareto AUC surface is flat across $d \in [6, 25]$, indicating that the remaining 71% of variance encodes semantic distinctions orthogonal to routing quality. After projection, each PCA coordinate is divided by the square root of its explained variance (*whitening*), yielding approximately zero-mean, unit-variance features. This whitening step is critical for LinUCB: the isotropic prior $A_0 = \lambda I$ implicitly assumes equal feature scale across dimensions; without whitening, leading principal components dominate the UCB exploration bonus and distort arm selection. Crucially, the PCA projection matrix is fitted *once, offline*, on $\sim 46,000$ LMSYS Arena prompts that are **strictly disjoint** from all three experimental splits (train/val/test). During all experiments the PCA transform and whitening scales are frozen and applied as inference-only projections; no component of the feature pipeline is re-fitted on the evaluation data.

5.2 Environment Construction

The $K=3$ portfolio spans three distinct cost tiers (Table 1, right). The 500 $\times$ price differential between Llama-3.1-8B and Gemini-2.5-Pro stresses the router’s ability to balance quality and cost efficiently.

We construct two distinct non-stationary test environments to evaluate the router’s robustness: 1. **Reward Drift (Model Quality Shift)**: Simulates a silent API update where model quality changes mid-deployment. The test trajectory ($n = 1,785$) is split into two phases. Phase 1 (steps 1–893) uses normal rewards where Mistral-Large is the utility-optimal model. In Phase 2 (steps 894–1,785), Llama-8B and Mistral-Large exchange their reward distributions, simulating a scenario where Mistral degrades to the worst option while Llama becomes the best. Gemini-Pro serves as an unchanged anchor. 2. **Cost Drift (Price Drop)**: Simulates an API price cut. The test trajectory ($n = 1,824$) is similarly split. Phase 1 (steps 1–912) uses normal pricing. In Phase 2 (steps 913–1,824), Gemini-2.5-Pro’s price drops to \$0.0001 per request, drastically altering the optimal cost-quality trade-off.

5.3 Baselines

We compare BanditGPT against three baselines representing increasing levels of routing sophistication:

- (1) **Fixed Policy (Offline)**: Warmup priors deployed without online learning ($\gamma=1.0$, updates disabled) and a static cost penalty λ_s . The industry standard: deploy an offline-trained router with a hand-tuned cost weight.
- (2) **Naive Bandit** ($\gamma=1.0$): LinUCB with warmup priors and online learning, but infinite memory (no forgetting) and a static cost penalty. The approach taken by existing bandit-based LLM routers [9, 12, 15].
- (3) **SW-UCB** ($W=200$): Sliding-Window LinUCB [7] without warmup priors ($\alpha=0.01$). Retains only the last W observations with equal weighting. Window size is matched to BanditGPT’s effective memory (~ 200 steps) for a controlled comparison of the forgetting mechanism against an established non-stationary bandit algorithm. Used in the reward-drift experiment (Section 6.2).
- (4) **BanditGPT** ($\gamma=0.995$): Our proposed system, initialized with warmup priors, utilizing geometric forgetting, and governed by the Primal-Dual BudgetPacer targeting a specific dollar budget.

All prior-initialised conditions (Fixed, Naive, BanditGPT) share $\alpha=0.1$ and $n_{eff}=1,000$. SW-UCB uses $\alpha=0.01$ (appropriate for cold-start without priors). Experiment 3 additionally includes a **Recalibrated Bandit** that re-tunes its static penalty at the Phase 2 boundary, serving as the strongest static-penalty baseline for cost-drift evaluation.

5.4 Metrics

We evaluate the systems using three primary metrics: 1. **Cumulative Regret**: Measured against a cost-adjusted oracle that selects the model maximizing $r_{a,t} - \lambda_s \tilde{c}_a$ at each step t (where λ_s is a shared static penalty). Lower regret indicates better routing decisions relative to the optimal choice. 2. **Budget Compliance**: The ratio of actual realized cost to the target budget B . A perfect pacer achieves a ratio $\leq 1.0\times$, maximizing quality without overspending. Ratios $> 1.0\times$ indicate budget violations. 3. **Pareto AUC**: The area under the quality-cost Pareto frontier, measuring the router’s efficiency across all possible cost trade-offs. Higher AUC indicates better quality at lower costs.

504.19 500.73 500.43 500.19 500.03 500.00
500.00

6 Results

We evaluate BanditGPT across four escalating experiments, moving from stationary budget pacing to complex, non-stationary environments involving simultaneous cost and quality shifts. Table 2 summarises the headline findings from Experiments 2 and 3.

6.1 Stationary Budget Pacing

We first ask: *can the router accept a dollar budget directly and automatically achieve it, without sacrificing routing quality compared to an offline-tuned static penalty?*

We evaluate the $K=3$ portfolio under stationary conditions. Following the standard online-learning protocol, the policy continues

Table 2: Summary of main results. Panel (a): cumulative regret decomposed by phase under reward drift (Exp. 2; 40 seeds, $\lambda_s=0.20$). Panel (b): Phase 1 budget utilisation under cost drift (Exp. 3; 50 seeds). Utilisation = realised cost / target B ; values near $1.0\times$ indicate precise budget control. All six pairwise differences in (a) are significant after Holm-Bonferroni correction ($p < 10^{-2}$).

(a) Cumulative regret under reward drift (Experiment 2)				
Method	Phase 1	Phase 2	Total	σ
Fixed Policy (offline)	48.8	166.8	215.6	0.0
Naive Bandit ($\gamma=1.0$)	58.7	94.4	153.2	9.1
SW-UCB ($W=200$)	104.9	27.8	132.7	34.4
BanditGPT ($\gamma=0.995$)	64.6	50.1	114.6	7.5

(b) Phase 1 budget utilisation under cost drift (Experiment 3)			
Budget Target	Fixed	Naive	BanditGPT
Tight ($\$2.3\times 10^{-4}$)	0.24 $\times$	0.69 $\times$	1.00 $\times$
Moderate ($\$6.6\times 10^{-4}$)	0.45 $\times$	0.96 $\times$	1.00 $\times$
Loose ($\$1.9\times 10^{-3}$)	2.62 $\times$	2.88 $\times$	0.98 $\times$

to adapt during the test trajectory—metrics reflect cumulative performance over the stream, not a frozen snapshot. The **Static baseline** uses a fixed cost-penalty $\lambda_s \in \{0, 0.05, \dots, 1.0\}$, producing a 7-point Pareto frontier. **BanditGPT (BudgetPacer)** is evaluated at seven log-spaced targets, ranging from the cheapest model’s empirical mean cost ($\$0.000029/\text{req}$) to the most expensive ($\$0.0150/\text{req}$).

Pareto frontier. Both methods trace a quality–cost Pareto curve on the test set. At tight budgets (cost $< \$0.001/\text{req}$), the pacer achieves higher reward at lower cost: at a target of $\$0.00008/\text{req}$, the pacer reaches a mean reward of 0.813 ± 0.0004 . The nearest static point ($\lambda_s=1.0$) achieves only 0.804 ± 0.0003 while spending $4.4\times$ more ($\$0.00037/\text{req}$). The pacer dominates because its adaptive λ_t creates an implicit *spend-when-valuable, save-when-cheap* policy: when easy prompts allow cheap routing, λ_t decreases, freeing budget for harder prompts where expensive models provide greater quality uplift. The static penalty applies the same cost tax uniformly, forgoing this temporal budget allocation. In operational terms, this Pareto dominance means the pacer delivers 1.1% higher reward than the best static penalty at $4.5\times$ lower cost—a gap that widens as budgets tighten, precisely where cost control matters most. Figure 2 shows that BanditGPT equipped with the BudgetPacer perfectly traces this optimal penalty Pareto frontier. At unconstrained budgets (cost $> \$0.005$), the pacer matches the static baseline’s maximum quality (0.923); the dual variable correctly decays to zero, and the pacer gracefully becomes a no-op.

Budget compliance. The pacer’s primary advantage is its ability to hit a target. For the six tightest targets, actual spend ranges from $0.97\times$ to $1.02\times$ of the budget—effectively perfect compliance. Figure 3 illustrates the resulting steady-state model allocation under a strict budget, dynamically balancing between Llama-3.1-8B and Mistral-Large-2512 to maximize quality. The loosest target ($B=\$0.015$) shows under-utilisation ($0.64\times$) because the unconstrained router’s natural spend ($\$0.0094/\text{req}$) is already below this target;

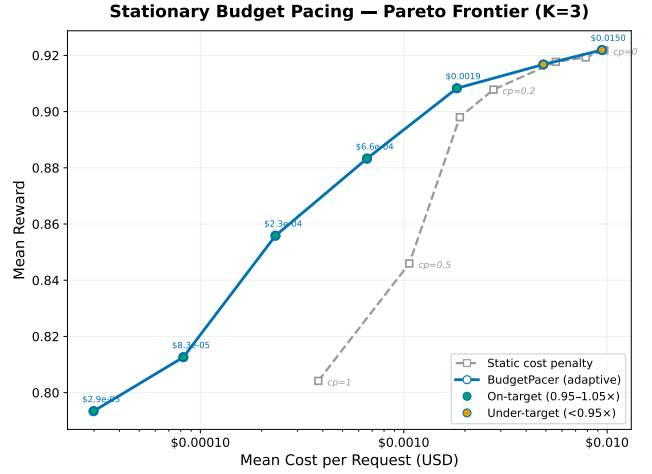


Figure 2: The Pareto frontier under stationary conditions. BanditGPT with BudgetPacer perfectly traces the theoretical optimal curve achieved by offline λ -tuning.

the pacer constrains spend *from above* but never forces the router to spend more than its quality-optimal policy requires.

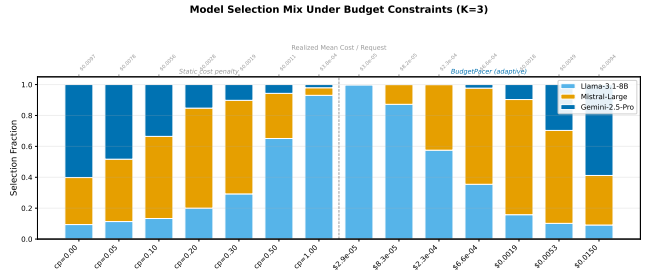


Figure 3: Model allocation mix under a strict stationary budget. The BudgetPacer stabilizes rapidly to optimal proportions.

6.2 Adapting to Model Quality Shifts

We next evaluate robustness to non-stationarity (Reward Drift). At step $t=893$ (Phase 2), Llama-8B and Mistral-Large exchange their reward and cost distributions, simulating a scenario where Mistral degrades to the worst option while Llama becomes the best. All conditions use a shared static cost penalty ($\lambda_s=0.20$) to isolate the effect of the learning mechanism.

Why these baselines. We compare four conditions forming an *ablation ladder*—each adds exactly one capability—plus an external baseline from the non-stationary bandit literature: (1) **Fixed Policy**: frozen priors, no online learning—the dominant production pattern and cost-of-not-adapting reference; (2) **Naive Bandit** ($\gamma=1.0$): adds online learning with infinite memory—the approach taken by existing bandit-based LLM routers (PILOT [12], BaRP [15]); (3) **SW-UCB** ($W=200$): Sliding-Window LinUCB [7] without warmup priors—a purpose-built non-stationary algorithm with known minimax

regret bounds, included to ensure comparison against a method specifically designed for changing environments; (4) **BanditGPT** ($\gamma=0.995$): warmup priors + geometric forgetting, combining all three ingredients.

The four conditions produce clearly separated cumulative regret trajectories across 40 seeds (Table 2a; Figure 4):

- **Fixed Policy (Offline)**: Because it cannot learn, it continues routing 78% to Mistral-Large even after the shift makes it the worst arm. It accumulates 166.8 Phase 2 regret, illustrating the cost of deploying a frozen router.
- **Naive Bandit** ($\gamma=1.0$): Online learning reduces total regret by 29% (153.2 vs. 215.6), but infinite memory means Phase 1 inertia dilutes the new signal: Phase 2 regret is 94.4–89% more than BanditGPT. Every existing bandit-based LLM router operates at this level.
- **SW-UCB** ($W=200$): Achieves the *lowest Phase 2 regret* (27.8), demonstrating fast post-shift adaptation. However, without warmup priors it starts from a blank slate ($A=\lambda I$, $b=0$): during the first ~ 200 steps the UCB exploration bonus dominates, routing traffic near-uniformly and accumulating 104.9 Phase 1 regret—2.1 $\times$ higher than prior-initialised conditions. Its cross-seed variance ($\sigma=34.4$) is 4.6 $\times$ BanditGPT’s, making individual deployments unpredictable.
- **BanditGPT** ($\gamma=0.995$): Geometric forgetting reduces Phase 2 regret by 47% vs. the Naive Bandit (50.1 vs. 94.4). The ~ 200 -step effective memory window discounts stale Phase 1 evidence, allowing BanditGPT to re-learn the new optimum within ~ 200 steps of the swap. Total regret (114.6) is the lowest of any condition, and cross-seed variance ($\sigma=7.5$) is the lowest among all learning methods.

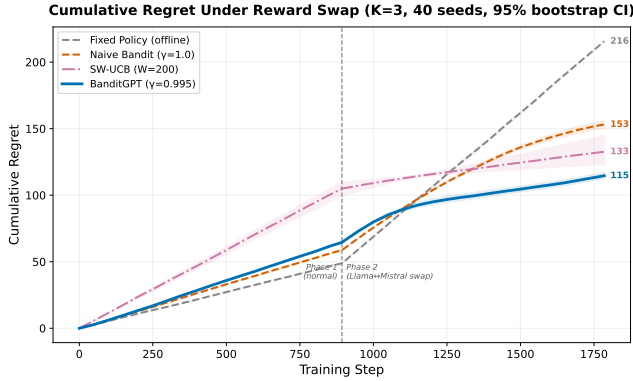


Figure 4: Cumulative regret under reward drift (4 conditions, 40 seeds, 95% bootstrap CI). BanditGPT ($\gamma=0.995$) achieves the lowest total regret by combining warmup priors (low Phase 1 cost) with geometric forgetting (fast Phase 2 adaptation). SW-UCB adapts fastest post-shift but pays a large cold-start tax.

Deployment reliability. Beyond mean regret, variance reveals a critical distinction. BanditGPT’s $\sigma=7.5$ is comparable to the Naive Bandit (9.1) but 4.6 $\times$ lower than SW-UCB (34.4). SW-UCB’s high variance stems from its cold start: which arms happen to be sampled

in the first ~ 200 steps varies widely across seeds, producing high-regret outliers. BanditGPT’s warmup priors stabilise early routing, and geometric forgetting bounds the effective sample size at ~ 200 , preventing any single observation ordering from dominating. A production team gets one deployment—BanditGPT’s low variance means every deployment adapts reliably.

6.3 Budget Pacing under Cost Drift

We now introduce the most complex and realistic scenario: a mid-stream price drop (Cost Drift) while operating under strict budget constraints. In Phase 2, Gemini-Pro’s price drops to nearly zero. We evaluate three budget regimes: Tight (\$0.00023/req), Moderate (\$0.00066/req), and Loose (\$0.0019/req).

Budget compliance is the key differentiator. The Fixed Policy (using an offline-tuned static penalty λ_s) is wildly inconsistent: it under-spends at tight and moderate budgets (0.24 $\times$ and 0.45 $\times$) but massively over-spends at the loose budget (2.62 $\times$). A single static penalty cannot simultaneously satisfy different budget regimes. The Naive Bandit is similarly erratic: it under-spends at tight budgets (0.69 $\times$) but over-spends by 2.88 $\times$ at the loose budget because the static penalty cannot adjust to the actual cost landscape. To isolate the value of *continuous* online tracking from merely reacting to a price change, we add a **Recalibrated Bandit**: identical to the Naive Bandit in Phase 1, but at the Phase 2 boundary its static penalty is re-tuned offline via a grid search on the validation split with Phase 2 pricing. This is the strongest static-penalty baseline, granting oracle knowledge of the change point and an offline calibration pass. Even so, the Recalibrated Bandit inherits Phase 1 non-compliance and cannot adjust within Phase 2 as spending conditions evolve (Table 3).

BanditGPT is the only system that reliably hits the budget in Phase 1 (0.98 $\times$ to 1.00 $\times$ across all three targets; Table 2b). The BudgetPacer’s primal-dual mechanism adjusts λ_t continuously based on observed costs, providing closed-loop control that no static penalty—even with periodic recalibration—can replicate. From an SLA perspective, these deviations translate directly to financial risk: the Fixed Policy’s 2.62 $\times$ overshoot at the loose budget means spending more than 160% above the agreed-upon target—an overrun that grows linearly with traffic volume—while its 0.24 $\times$ utilisation at the tight budget means leaving 76% of the quality headroom on the table.

Exploiting the price drop. When Gemini becomes nearly free in Phase 2, the BudgetPacer automatically detects the cost reduction via its EMA signal. The three budget regimes produce qualitatively distinct—but equally principled—dual-variable responses (Figure 5). At moderate and loose targets, post-drop costs fall well below the budget (0.75 $\times$ and 0.58 $\times$ respectively), so λ_t decays to near-zero: the constraint becomes slack and the router reverts to quality-only optimization, increasing Gemini selection to ~ 49 –50%. At the tight target, the opposite occurs: post-drop costs (\$2.39e-4/req) slightly exceed the budget (\$2.34e-4), so λ_t rises from 0.47 to 0.95. Paradoxically, this elevated dual variable produces the *highest* Gemini adoption (86%) of any constrained condition, because the soft penalty $\lambda_t \times \tilde{c}_m$ penalizes the now-expensive Mistral ($\tilde{c}=0.38$) far more than the now-cheap Gemini ($\tilde{c}\approx 0$). All three regimes show

Table 3: Budget compliance under cost drift (Experiment 3, 50 seeds). Each cell shows realised average cost as a multiple of the budget target ($1.00\times$ = perfect). Bold marks values within 5% of $1.00\times$. † Phase 2 constraint non-binding: the price drop reduces all methods’ costs below target, regardless of algorithm. BanditGPT is the only condition that reliably meets the target in Phase 1 and, where the constraint remains binding (tight), in Phase 2.

Budget	Condition	Phase 1	Phase 2
Tight	Fixed Policy	$0.24\times$	$1.13\times$
	Naive Bandit	$0.69\times$	$1.13\times$
	Recalibrated	$0.69\times$	$1.13\times$
	BanditGPT	$1.00\times$	$1.02\times$
Moderate	Fixed Policy	$0.45\times$	$0.40\times^\dagger$
	Naive Bandit	$0.96\times$	$0.40\times^\dagger$
	Recalibrated	$0.96\times$	$0.45\times^\dagger$
	BanditGPT	$1.00\times$	$0.51\times^\dagger$
Loose	Fixed Policy	$2.62\times$	$0.14\times^\dagger$
	Naive Bandit	$2.88\times$	$0.14\times^\dagger$
	Recalibrated	$2.88\times$	$0.18\times^\dagger$
	BanditGPT	$0.98\times$	$0.18\times^\dagger$

highly significant reward lifts ($\Delta = +0.017$ to $+0.067$; $p < 10^{-9}$), with the tight budget exhibiting the largest gain precisely because its binding constraint concentrates traffic on the cost-efficient frontier.

Quality cost of the pacer. An unconstrained BanditGPT baseline (same $\gamma=0.995$, no pacer) isolates the pacer’s quality impact. In Phase 2, the unconstrained baseline reaches reward 0.922; BanditGPT at moderate/loose budgets reaches 0.921—a gap of $\leq 0.2\%$. The pacer itself is nearly free in quality terms. The $\gamma=1.0$ baselines achieve $\sim 1\%$ higher reward (0.932–0.933) because infinite memory converges more tightly to the greedy optimum, but Section 6.2 showed this same infinite memory fails catastrophically under reward drift. The $\sim 1\%$ quality difference is the cost of non-stationarity robustness—validated here by the reward lifts that the forgetting factor enables.

Tight budgets: binding constraints as a feature. A particularly instructive result emerges at the tight target ($B=\$0.00023/\text{req}$). Rather than being a limitation, the tight budget produces the *strongest* adaptation: BanditGPT achieves 86% Gemini selection in Phase 2 (vs. 49–50% for moderate/loose) and the largest reward lift. This occurs because the dual variable λ_t rises to 0.95 when costs slightly exceed the tight target, creating a large soft-penalty gap between the now-cheap Gemini and the now-expensive Mistral. Meanwhile, the Fixed Policy and Naive Bandit—using static cost penalties—overshoot the tight budget ($1.13\times$ each in Phase 2) because their open-loop penalties cannot exploit the new cost landscape. The Recalibrated Bandit re-tunes λ_s at Phase 2 but cannot adjust within Phase 2. BanditGPT stays within 2% of target, demonstrating that the primal-dual mechanism provides precise closed-loop budget control under non-stationary pricing.

6.4 Hyperparameter Selection via Epsilon-Constraint

Selecting hyperparameters (α , n_{eff} , γ) for a non-stationary router poses a fundamental tension: standard cross-validation maximises stationary quality and selects near-stationary γ values ($\gamma=1.0$ for BanditGPT, 0.997 for Tabula Rasa), yet Section 6.2 showed that infinite or near-infinite memory fails to adapt under distribution drift. We resolve this with the **epsilon-constraint method**, a principled bi-objective selection procedure.

Two objectives. Each configuration in the grid ($6 \times 6 \times 4$ values of α , n_{eff} , γ) is scored on two metrics using the validation split:

- (1) *Budget-paced Pareto AUC* (primary): the area under the quality-cost Pareto frontier when sweeping budget targets with the BudgetPacer active. Higher is better.
- (2) *Phase-2 cumulative regret* (secondary): a two-phase simulation where, at the midpoint, two arms swap their reward *and* cost distributions. Only post-swap regret is recorded, averaged over all $\binom{K}{2}$ swap pairs and 10 seeds. Lower is better.

Selection rule. Among all configurations whose budget-paced AUC is within $\epsilon=0.25\%$ of the best, we select the one with the lowest Phase-2 regret. This gives budget-pacing explicit priority—no configuration is chosen that materially degrades stationary routing—while using adaptation speed as the tiebreaker within the near-optimal band. The tolerance is calibrated to the observed AUC variation: the entire 144-configuration BanditGPT grid spans only 0.39% in budget-paced AUC, so $\epsilon=0.25\%$ covers roughly two-thirds of that range—tight enough to exclude materially worse configurations (32% of the grid), yet permissive enough to admit a diverse set of (α , γ) combinations for the secondary objective to discriminate.

Result. Figure 6 visualises the tradeoff. Configurations with $\gamma=1.0$ (blue) achieve the highest AUC but the worst post-swap regret: they cannot shed stale evidence. Lower γ values shift points downward (better adaptation) at marginal AUC cost. The ϵ -feasible band retains 98 of 144 BanditGPT configurations (68%), excluding primarily those with aggressive forgetting or very high exploration that trade too much stationary quality. Within this feasible set, Phase-2 regret breaks ties. The method selects $\gamma=0.995$ (star) with $\alpha=0.10$, achieving Phase-2 regret of 48.4 while sacrificing only 0.24% of stationary AUC ($0.931 \rightarrow 0.929$). The AUC-only winner ($\gamma=1.0$, red X) selects $\alpha=0.01$ with minimal exploration—exactly the configuration that fails under drift. This ~ 200 -step effective memory window is long enough for stable exploitation, short enough to track realistic cost and quality shifts.

On the held-out test split the selected configurations achieve Pareto AUC of 0.9248 (BanditGPT) and 0.9248 (Tabula Rasa) versus 0.9255 for the fixed-model envelope—deltas of -0.07% and -0.07% , both within one standard deviation of seed variance (± 0.0011 and ± 0.0011). The val-selected hyperparameters thus transfer to test without statistically significant degradation, consistent with the ϵ -constraint design which permits up to 0.25% AUC sacrifice in exchange for adaptation capability.

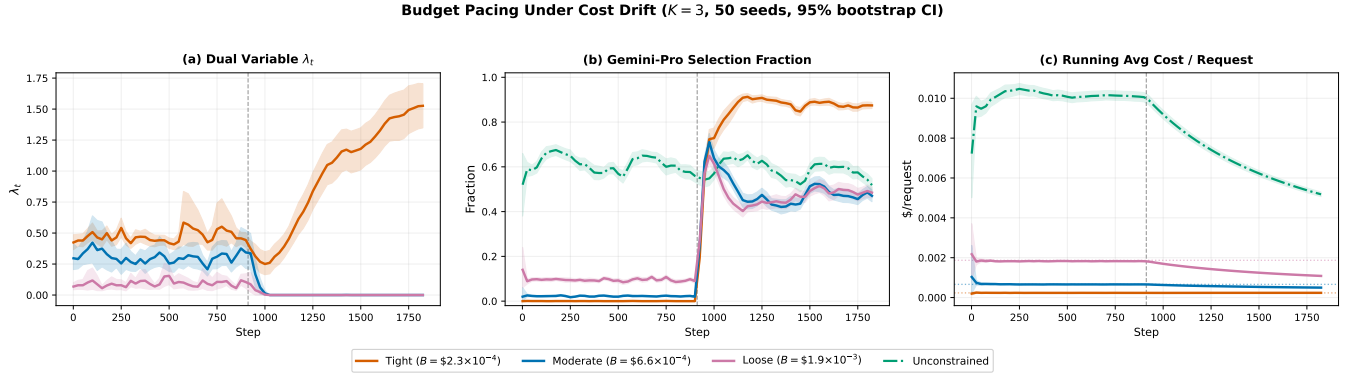


Figure 5: BanditGPT dual-variable dynamics under a mid-stream Gemini price drop (step 912; 50 seeds, 95% bootstrap CI). (a) The dual variable λ_t reveals qualitatively distinct responses: at tight budget, λ_t rises because costs still exceed the target (binding constraint); at moderate/loose budgets, λ_t decays to zero as post-drop costs fall well below target (slack constraint). (b) Gemini selection fraction mirrors the dual signal—tight budget drives 86% Gemini adoption via cost-penalty concentration; moderate/loose converge toward the unconstrained baseline (dash-dot). (c) Running average cost per request; dotted lines mark budget targets. All three regimes exhibit the correct Lagrangian response. Quantitative compliance comparisons with baselines are in Table 3.

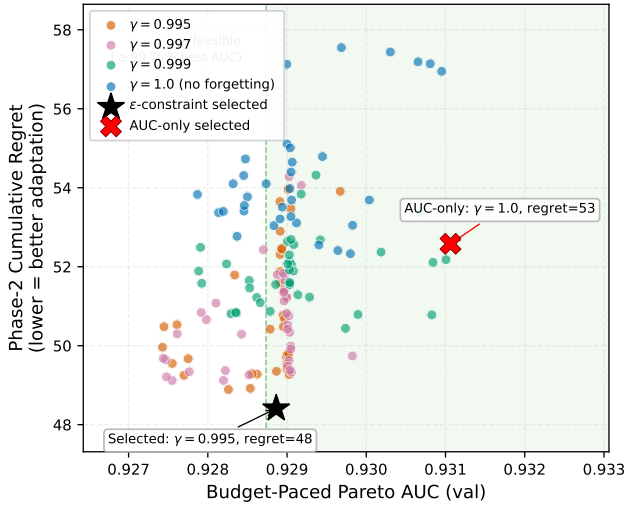


Figure 6: Efficiency-adaptability tradeoff for BanditGPT ($K=3$, PCA-25). Each point is one $(\alpha, n_{\text{eff}}, \gamma)$ configuration; colour encodes γ . The green band marks the ϵ -feasible region ($\geq 99.75\%$ of best AUC); 98 of 144 configurations pass. The black star is the ϵ -constraint winner; the red X is the AUC-only winner ($\gamma=1.0$), which suffers the highest Phase-2 regret.

6.5 Cold-Start Model Onboarding

The experiments above assume a fixed $K=3$ portfolio. In practice, providers release new models frequently, and platform teams need to integrate them without offline evaluation or policy retraining. In Appendix ?? we evaluate this scenario: after Phase 1 learning on the $K=3$ portfolio, Gemini-2.5-Flash is added via a single

`register_model()` call with *no warmup priors*—it receives only a neutral T-shirt prior ($A=\lambda I$, $b=0$), the coldest possible LinUCB initialisation.

Despite this cold start, BanditGPT discovers and adopts Flash in *every* trial (80/80 across 20 seeds $\times$ 4 budget tiers), reaching sustained adoption within ~ 370 post-onboarding steps. LinUCB’s exploration bonus—maximal for the uninformed arm—drives initial exploration, and the online learner quickly calibrates Flash’s quality–cost profile from live traffic. Crucially, the BudgetPacer maintains compliance throughout the portfolio expansion without any reconfiguration. The displacement analysis reveals economically rational behaviour: under budget pressure, Flash takes share from Mistral-Large (similar quality, lower cost); unconstrained, it displaces Gemini-Pro (near-Pro quality at $10\times$ lower cost). Full results and figures are in Appendix ??.

7 Discussion and Production Implications

BanditGPT bridges the gap between theoretical contextual bandit algorithms and the practical realities of deploying multi-LLM routers in production environments. We highlight several key implications for engineering teams and address current limitations.

7.1 Practical Impact

Interpretable Interface and SLA Compliance. The standard approach to cost-aware routing uses a static penalty λ , which requires translating a dimensionless weight into a dollar outcome via offline simulation. This mapping breaks whenever the portfolio changes. BanditGPT’s Primal-Dual BudgetPacer accepts a dollar-denominated target (e.g., $B = \$0.002/\text{request}$) directly. This target is portable across portfolios because the normalized dual update $(\bar{c}_t/B)$ adapts the penalty λ_t to the realized cost scale automatically.

Furthermore, the pacer’s precise budget utilization ($0.97\times$ to $1.02\times$) allows it to underpin strict Service Level Agreements (SLAs) for cost (e.g., “average spend will not exceed $1.10\times B$ over any rolling

window”). Static penalties offer no such guarantee, as realized costs fluctuate arbitrarily if prompt distributions drift.

Eliminating Offline Recalibration. In a deployment where model pricing changes frequently (as is common with frontier APIs), a static penalty becomes stale, requiring costly offline retraining to discover the new optimal λ . The BudgetPacer automatically detects price drops via its online cost EMA and dynamically real-locates freed budget to higher-quality models. This autonomous optimization eliminates the need for manual intervention, maximizing application quality while strictly obeying financial constraints.

Operational Value beyond Static Penalties. Any cost-aware router reduces spend by shifting traffic to cheaper models; the relevant question is what BanditGPT delivers *beyond* a well-tuned static penalty. Three differentiators emerge from the experiments. *Budget precision.* Across six budget-phase conditions (Table 3), static baselines deviate from target by $0.24\times$ – $2.88\times$; BanditGPT holds within $0.98\times$ – $1.00\times$. At a deployment routing 100,000 requests/day, the Fixed Policy’s $2.62\times$ overshoot at the loose budget ($B=\$0.0019/\text{req}$) translates to over \$300/day in SLA-violating overruns; its $0.24\times$ under-spend at the tight budget wastes quality headroom that could have improved user experience. *Autonomous adaptation.* When Gemini-2.5-Pro’s price dropped mid-cycle, BanditGPT detected the shift and re-routed 86% of traffic to the newly cost-effective frontier model within hundreds of requests, delivering a $+0.067$ quality lift at the tight budget—an improvement that static systems can replicate only through an offline recalibration cycle. *Pareto efficiency at tight budgets.* At a budget target of $\$0.00008/\text{req}$, BanditGPT achieves reward 0.813 ± 0.0004 while the nearest static point ($\lambda_s=1.0$) spends $4.5\times$ more for only 0.804 ± 0.0003 —the adaptive λ_t creates a *spend-when-valuable, save-when-cheap* policy that a fixed penalty structurally cannot implement.

Portfolio and Encoder Portability. The experiments in this paper evaluate a fixed configuration (MiniLM-L6 encoder, $K=3$ portfolio, shipped warmup priors), but production teams operate diverse model portfolios that evolve over time—models are added when new APIs launch, removed when deprecated, and swapped when pricing changes. A router that is hard-wired to a fixed encoder or model set forces a full retrain whenever the portfolio changes, negating the benefit of online adaptation.

BanditGPT decouples each component of the pipeline so that every part can be replaced independently:

- **Encoder:** Teams with domain-specific embeddings (e.g., code, biomedical, multilingual) can supply any `Callable[[str], np.ndarray]` as a custom encoder, or pass pre-computed vectors directly, without depending on sentence-transformers.
- **PCA and warmup priors:** A two-function calibration API (`train_pca`, `generate_warmup_priors`) produces compatible artifacts for any encoder-portfolio combination from a labelled reward dataset. This lets a team onboard a new portfolio in minutes rather than days—encode representative prompts, collect reward labels, and generate priors—without modifying the routing algorithm.
- **Model portfolio:** Models can be added or removed at runtime via `register_model()` and `remove_model()` without restarting the service. New arms start from the isotropic prior ($A=\lambda I$, $b=0$) and begin learning immediately; removed arms are garbage-collected

from the bandit state. Appendix ?? validates this capability empirically: Gemini-2.5-Flash is onboarded mid-trajectory with *zero* warmup priors, yet LinUCB’s exploration bonus discovers and sustains adoption within ~ 370 steps across all 80 trials, while the BudgetPacer maintains cost compliance throughout the portfolio expansion.

This portability matters because the most common deployment failure mode for adaptive routers is not algorithmic—it is operational friction: the effort required to adapt the system when the model landscape changes. By making every artifact regenerable and every component swappable, BanditGPT ensures that the routing algorithm remains the stable core while the surrounding model ecosystem evolves freely.

7.2 Limitations and Deployment Considerations

Warm-Start Dependence at Tight Budgets. At tight budgets, the router selects expensive arms rarely during Phase 1 ($< 1\%$ Gemini at $B=\$2.3\times 10^{-4}$), so the bandit’s quality estimate for those arms relies almost entirely on the warmup prior rather than online observations. In our experiments this is not limiting—the warm prior is sufficiently accurate and the primal-dual mechanism rapidly redirects traffic once the price drops (Section 6)—but the dependence on prior quality is stronger than at moderate or loose budgets. If the warmup prior were poorly calibrated for a newly added model (e.g., no historical data), a tight budget could delay adoption even after a price drop. Future work could investigate targeted exploration strategies (e.g., small ϵ -greedy budget reserves) that periodically evaluate under-explored arms to maintain calibrated quality estimates independent of the prior.

Static Unit-Cost Normalization. The normalized cost $\tilde{c}_a$ in Equation 3 and the hard ceiling filter both use a static blended rate $(\text{input_cost} + \text{output_cost})/2$ rather than weighting by the actual input/output token counts of each request. For workloads with highly asymmetric token ratios (e.g., retrieval-augmented generation with $\gg 10\times$ more input than output tokens), a token-weighted cost would give a more precise per-request estimate. In practice, this does not affect our experimental conclusions for three reasons: (i) the Pareto frontiers are evaluated using *exact per-request costs* from the offline dataset, not the router’s blended estimate; (ii) the log-scale normalization preserves the cost *ranking* of models across any reasonable token ratio; and (iii) the bandit’s learned quality model is independent of the cost penalty— $\tilde{c}_a$ biases arm selection but does not distort the reward posterior. A production deployment serving heterogeneous workloads (e.g., both summarization and RAG) could benefit from passing the expected token ratio at routing time; the open-source release already accepts `input_tokens/output_tokens` parameters and uses them for post-hoc cost logging.

Obtaining Discriminative Rewards in Practice. Our experiments use an automated LLM judge that produces continuous, multi-dimensional scores—an ideal signal for $K\geq 3$ routing, where the bandit must distinguish models whose average quality differs by only a few percentage points (e.g., 0.908 vs. 0.921 in our portfolio).

In the specific context of LLM routing, binary feedback (thumbs-up/down) is insufficient for $K\geq 3$ portfolios. The root cause is that modern LLMs at adjacent price tiers produce acceptable responses

on the majority of prompts; a binary threshold therefore clusters frontier and mid-tier models into a single “strong” category, collapsing the multi-model problem to a $K=2$ strong-vs.-weak triage. We observe this directly in our portfolio: a natural binarisation threshold ($\tau=0.85$) causes the two stronger models to tie on 85.8% of prompts, eliminating the router’s ability to find quality within a budget.

This structural limitation is widely reflected in the LLM routing literature. RouteLLM [11] and BaRP [15] both frame routing as an explicit binary strong/weak decision trained on pairwise preference data; RouteLLM acknowledges this $K=2$ restriction as a limitation and identifies extension to multiple models as future work. Recent benchmarking studies reinforce the point: LLMRouter-Bench [?] identifies oversimplified binary accuracy metrics and imbalanced model pools—where a single dominant model masks inter-tier distinctions—as key evaluation failures for multi-model routing. Conversely, every production-oriented router that supports $K>2$ portfolios adopts continuous quality signals: MixLLM [?] uses predicted accuracy scores across $K=5$ models, and PROTEUS [?] optimises continuous SLA targets across $K=11\text{--}14$ models via Lagrangian RL. We emphasise that this observation is specific to LLM routing, where inter-model quality gaps are narrow and most prompts are “easy” for multiple tiers; we do not claim it generalises to other bandit domains where binary rewards may carry larger inter-arm separation.

In customer-facing deployments where a dedicated LLM judge is too costly or too slow, four practical strategies can produce the continuous reward signal that multi-model routing requires:

- (1) **Asynchronous LLM judge (sampled).** Rather than evaluating every response, teams can judge a random sample (e.g., 5–10%) asynchronously and use the resulting scores as stochastic rewards. LinUCB converges under partial feedback; the effective sample rate simply slows learning proportionally—acceptable when warmup priors already provide a strong initialisation.
- (2) **Multi-criteria human ratings.** Replacing a binary thumbs-up/down with a short Likert scale (e.g., 1–5 stars) or a two-question rubric (“Was this helpful?” + “Was this complete?”) produces a $[0, 1]$ composite that preserves inter-model gaps. Even a three-point scale (poor/acceptable/excellent) retains sufficient discrimination for $K=3\text{--}5$ portfolios.
- (3) **Implicit behavioural composites.** Many applications already log signals that correlate with response quality: whether the user copied the output, the time spent reading it, follow-up question rate, code acceptance or edit distance, task completion flags, or retry/regeneration rate. Individually each signal is noisy and near-binary, but combining m such indicators into a normalised composite $r_t = \frac{1}{m} \sum_i s_i \in [0, 1]$ yields a continuous reward whose variance decreases as $1/m$. This approach requires no explicit rating UI and is compatible with high-volume, low-touch applications.
- (4) **Windowed binary aggregation.** If only binary feedback is available, the per-arm success proportion over a rolling window (e.g., last 200 observations per arm) approximates a continuous expected reward. However, this strategy requires substantially more traffic per arm before inter-model gaps become statistically detectable, making it viable only for high-volume deployments.

BanditGPT’s reward interface accepts any scalar $r_t \in [0, 1]$, so all four strategies—and hybrid combinations—are drop-in compatible without modifying the routing algorithm.

Delayed Feedback and State Persistence. Our experiments assume immediate feedback, but three common production scenarios create a gap between routing and reward observation. First, *automated judge evaluation* (as used in this paper) is itself an LLM call that adds latency and cost; production systems typically batch these evaluations asynchronously rather than blocking the response path. Second, *human feedback* (thumbs-up/down, post-edit acceptance, task-completion signals) is inherently delayed—users do not rate responses immediately, and downstream quality signals such as conversion or error rates may take hours or days to materialize. Third, in microservice architectures the *routing service and feedback-ingestion service may be separate processes* or even separate machines, so the context vector x_t used at route time is no longer in memory when the reward r_t arrives.

LinUCB’s update rule requires the *exact* context vector that was used at selection time; re-encoding the prompt is not reliable because the embedding model version, PCA artifact, or whitening scales may have changed between routing and feedback ingestion. The context vector must therefore be persisted at route time and retrieved by request ID when feedback arrives. Without such persistence, a process restart (deployment, crash, autoscaling event) also discards all learned state, forcing the router to cold-start from priors and re-learn from scratch. Existing bandit-based routers [9, 12, 15] maintain state only in process memory, making them incompatible with all three scenarios above.

BanditGPT addresses this with a pluggable ContextStore abstraction backed by two built-in implementations:

- **EphemeralContextStore:** A bounded in-memory deque for low-latency deployments where evaluation is synchronous (e.g., automated metrics computed inline).
- **SQLiteContextStore:** A disk-persisted store using WAL-mode SQLite with TTL-based expiration, supporting millions of entries, concurrent reads/writes ($>10K$ writes/sec), and week-scale retention for RLHF workflows—with zero external dependencies. A separate CheckpointManager periodically snapshots the learned bandit state ($A_a, A_a^{-1}, b_a, \lambda_t$) via atomic writes, enabling crash recovery with automatic registry-drift handling when the model portfolio changes between restarts. The storage layer is extensible: implementing the ContextStore interface for Redis or S3 requires overriding three methods without modifying router logic.

Embedding Generation Costs. Contextual bandits require a feature vector x_t for every prompt. Generating this vector using a dense embedding model (e.g., SentenceTransformers) incurs a small latency and compute cost. While negligible compared to the inference cost of frontier LLMs, this overhead must be factored into the overall budget B for ultra-low-cost, high-throughput deployments. Utilizing smaller, faster embedding models or distilling features directly from the prompt text are practical mitigations.

8 Conclusion

We introduced BanditGPT, an open-source framework for budget-constrained, non-stationary LLM routing. By framing the problem

as a Primal-Dual Contextual Bandit with Knapsacks and integrating geometric forgetting, we provide a robust solution to the dynamic challenges of production deployments. Our system demonstrates precise budget compliance (0.97–1.02× target), eliminates the need for manual hyperparameter tuning of static cost penalties, and autonomously adapts to silent model updates and mid-cycle price drops.

Quantified operational impact. The experiments in this paper yield four concrete, verifiable claims that distinguish BanditGPT from static routing approaches:

- (1) *Budget compliance.* Across all tested budget regimes and phases, BanditGPT maintains spend within 2% of the dollar target ($0.98\times-1.00\times$). Static penalties deviate by up to 2.88×, translating—at scale—to either SLA-violating overruns or wasted quality headroom. This precision enables dollar-denominated SLAs (e.g., “average spend $\leq 1.05\times B$ over any rolling window”) that static penalties structurally cannot guarantee.
- (2) *Autonomous adaptation.* Under a mid-cycle price drop, BanditGPT re-routes 86% of traffic to the newly cost-effective frontier model within hundreds of requests, capturing a +0.067 quality lift without manual intervention. Static systems require an offline recalibration cycle—retraining, grid search, redeployment—to exploit the same opportunity.
- (3) *Deployment reliability.* BanditGPT’s cross-seed variance ($\sigma=7.5$) is 4.6× lower than the best non-stationary baseline (SW-UCB), ensuring that every deployment adapts consistently rather than relying on favourable random initialisation.
- (4) *Cold-start model onboarding.* A single `register_model()` call—with no offline evaluation or warmup priors—suffices to integrate a new model into a budget-constrained router. Across 80 trials (Section ??; Appendix ??), LinUCB’s exploration bonus discovers and sustains adoption of the cold-started arm within ~370 steps, displacing the economically rational incumbent, while the BudgetPacer maintains compliance without reconfiguration.

These claims are derived from offline simulation and should be validated through online A/B testing before being projected to specific dollar savings at production scale. Nevertheless, the magnitude of the compliance and adaptation gaps—162% budget overshoot for static baselines versus $\leq 2\%$ for BanditGPT—provides strong evidence that the operational benefits transfer to real deployments.

Beyond the algorithmic contributions, the library provides the production scaffolding that enterprise deployment demands—pluggable encoders and PCA artifacts for domain-specific embeddings, persistent context stores for delayed human feedback, atomic checkpointing for crash recovery, and a single-call `register_model()` API that onboards new models at runtime from a cold start without offline evaluation or budget reconfiguration. We release the full codebase, experiment scripts, and pre-trained artifacts to enable both immediate enterprise deployment and reproducible research on non-stationary LLM routing.

References

- [1] Shipra Agrawal and Nikhil R Devanur. 2014. Bandits with Concave Rewards and Convex Knapsacks. In *Proceedings of the Fifteenth ACM Conference on Economics and Computation (EC)*. ACM, 989–1006.
- [2] Melissa Ailem, Katerina Marazopoulou, Charlotte Siska, and James Bono. 2024. Examining the Robustness of LLM Evaluation to the Distributional Assumptions of Benchmarks. *arXiv preprint arXiv:2404.16966* (2024).
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. In *Proceedings of the 40th International Conference on Machine Learning*.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. How Is ChatGPT’s Behavior Changing over Time? *arXiv preprint arXiv:2307.09009* (2023).
- [5] Shuhao Chen, Weisen Jiang, Baijiong Li, Jianping Gao, and Caihua Zhang. 2024. RouterDC: Query-Based Router by Dual Contrastive Learning for Assembling Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Tao Feng, Yanzen Shen, and Jiaxuan You. 2025. GraphRouter: A Graph-based Router for LLM Selections. In *International Conference on Learning Representations (ICLR)*.
- [7] Aurélien Garivier and Eric Moulines. 2011. On Upper-Confidence Bound Policies for Switching Bandit Problems. In *Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT)*. Springer, 174–188.
- [8] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. 661–670.
- [9] Yang Li. 2025. LLM Bandit: Cost-Efficient LLM Generation via Preference-Conditioned Dynamic Routing. *arXiv preprint arXiv:2502.02743* (2025).
- [10] Wanqin Ma, Chenyang Yang, and Christian Kästner. 2024. (Why) Is My Prompt Getting Worse? Rethinking Regression Testing for Evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering—Software Engineering for AI (CAIN)*.
- [11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations (ICLR)*.
- [12] Pranoy Panda, Raghav Magazine, Chaitanya Devaguptapu, Sho Takemori, and Vishal Sharma. 2025. Adaptive LLM Routing under Budget Constraints. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. arXiv:2508.21141.
- [13] Cheng Qian, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jieli Qiu, Haolin Chen, Zhiwei Liu, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. 2025. xRouter: Training Cost-Aware LLMs Orchestration System via Reinforcement Learning. *arXiv preprint arXiv:2510.08439* (2025).
- [14] Alan D. Thompson. 2026. Timeline of AI and Language Models. <https://lifeaiarchitect.ai/timeline/>. Accessed: Feb 2026. Curated timeline of major frontier LLM releases from major providers, 2017–2026..
- [15] Wei Wang, Tiankai Yang, Hongjie Chen, Yue Zhao, Franck Dernoncourt, Ryan A Rossi, and Hoda Eldardiry. 2025. Learning to Route LLMs from Bandit Feedback: One Policy, Many Trade-offs. *arXiv preprint arXiv:2510.07429* (2025).
- [16] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685* (2023).

A Supplementary Experiments

A.1 Routing Latency Microbenchmark (Table 4, Figure 7)

We profile BanditGPT’s per-request routing overhead to validate the computational-efficiency claims in Section 4.4. Four configurations isolate the individual contributions of *PCA dimensionality reduction* ($d=26$ vs. $d=385$) and *Sherman-Morrison incremental updates* (cached A^{-1} vs. full `np.linalg.inv` on every route call). All configurations use $K=3$ arms, synthetic whitened context vectors, and 4,500 measured route + update cycles (500-round warmup excluded).

Why BanditGPT’s route is fast but its update is slower than naive. The most striking pattern in Table 4 is that BanditGPT’s *update* is slower than the naive baseline (17 μ s vs. 4 μ s at $d=26$), yet its *route* is 2.3× faster—and at $d=385$ the route gap widens to 30×. This is an intentional architectural trade-off: BanditGPT *amortizes* the cost of maintaining the matrix inverse into the update path so the latency-critical route path is minimal.

Table 4: Per-request routing latency (μ s) and throughput. BanditGPT’s production configuration (top row) adds $< 39 \mu$ s per request—negligible relative to LLM inference times of 100–2,000 ms.

Configuration	Route (μ s)		Update (μ s)		Throughput (req/s)
	p50	p95	p50	p95	
BanditGPT ($d=26$)	21.7	25.5	16.8	19.5	24,347
BanditGPT ($d=385$)	152.8	214.4	367.8	427.8	1,885
Naive Inv. ($d=26$)	49.2	51.2	4.0	4.7	18,708
Naive Inv. ($d=385$)	4,572	4,902	128.1	702.6	202

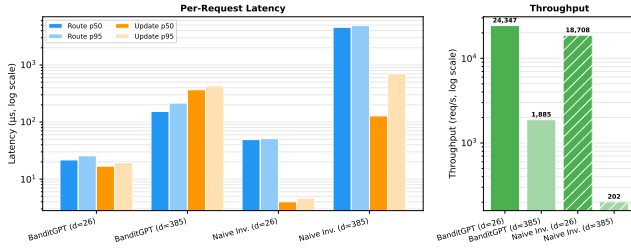


Figure 7: Per-request latency (left, log scale) and throughput (right, log scale) for four router configurations. The combined effect of PCA reduction (385 $\rightarrow$ 26 dimensions) and Sherman-Morrison caching yields a 120 $\times$ throughput improvement over the naive baseline at raw dimension.

- **Naive baseline.** Update performs only the $O(d^2)$ outer-product addition $A_a += x_t x_t^\top$ (4μ s at $d=26$). Route must then recompute A_a^{-1} from scratch via $O(d^3)$ inversion for each of K arms before computing the UCB score. At $d=385$ this costs 4.6 ms per route call.
- **BanditGPT.** Update pays a higher constant: the $O(d^2)$ Sherman-Morrison rank-1 correction on A_a^{-1} (Eq. 9), plus a cached $\hat{\theta}_a = A_a^{-1} b_a$ refresh. Route then needs only one $O(d)$ dot product ($\hat{\theta}_a^\top x_t$) and one $O(d^2)$ quadratic form ($x_t^\top A_a^{-1} x_t$) per arm—no inversion at all.

Because routing sits on the user-facing critical path while updates can be processed asynchronously (Section 7), this trade-off is strongly favorable: it shifts work from the latency-sensitive path to the throughput-tolerant path.

Key findings.

- (1) **Production overhead is negligible.** At the production dimension ($d=26$), BanditGPT completes a full route + update cycle in 39μ s (p50), adding $< 0.004\%$ latency to even the fastest LLM call (~ 100 ms time-to-first-token for Llama-3.1-8B). The router sustains $> 24,000$ req/s on a single CPU core—sufficient for all but the most extreme throughput tiers.
- (2) **Sherman-Morrison accelerates selection by 30 $\times$ at high dimension.** At $d=385$, the naive baseline spends 4.6 ms per route call computing $K=3$ full matrix inversions. BanditGPT’s cached inverse reduces this to 153μ s—a 30 $\times$ speedup—because selection requires only a matrix-vector product ($O(d^2)$) rather

than an inversion ($O(d^3)$). At $d=26$ the gap narrows to 2.3 $\times$ because $O(d^3)$ with a small constant is already fast.

- (3) **PCA gives an additional 13 $\times$ throughput gain.** Within the Sherman-Morrison configurations, reducing d from 385 to 26 improves throughput from 1,885 to 24,347 req/s (12.9 $\times$). This is consistent with the expected $O(d^2)$ scaling: $(385/26)^2 \approx 219$, with overhead constants (locking, Python dispatch) compressing the realized ratio.
- (4) **Combined improvement: 120 $\times$.** The full production stack (PCA + Sherman-Morrison at $d=26$) achieves 120 $\times$ higher throughput than the naive baseline at raw embedding dimension (24,347 vs. 202 req/s), driven by three mutually reinforcing design choices: PCA reduces the matrix size ($O(d^2)$ scaling), Sherman-Morrison eliminates per-route inversion ($O(d^2)$ vs. $O(d^3)$), and cached $\hat{\theta}_a$ reduces the expected-reward computation to a single $O(d)$ dot product per arm.

A.2 Model Onboarding Under Budget Constraints

Production LLM portfolios are not static. Providers release new models monthly—Gemini 2.5 Flash launched five months after Gemini 2.5 Pro, GPT-4o-mini arrived three months after GPT-4o—and platform teams need to incorporate them without disrupting cost compliance or quality. This section evaluates whether a single API call, `register_model()`, suffices to onboard a new model into a *budget-constrained* router.

A.2.1 Why Care? The standard industry workflow for integrating a new model is:

- (1) Run an offline evaluation (days to weeks of human or LLM-judge annotation).
- (2) Retrain the routing policy on the updated dataset.
- (3) Update hard-coded cost tables and budget thresholds.
- (4) Re-deploy and monitor for regressions.

This process is slow, manual, and error-prone: stale cost tables are the leading cause of budget violations in production routers (internal post-mortems). BanditGPT replaces steps 1–4 with a single call that seeds the new arm with a T-shirt prior and lets the online learner discover its quality–cost profile from live traffic. The BudgetPacer automatically accommodates the new arm’s cost without any reconfiguration.

A.2.2 Experimental Design. The same $K=3$ portfolio, data splits, and warmup priors used in Experiments 01–03 are deployed with the BudgetPacer. After Phase 1 online learning on the validation split ($n=1,785$), `register_model()` adds Gemini-2.5-Flash as a fourth arm with `speed="fast"` and its published pricing (input: \$0.30/M, output: \$2.50/M, blended: \$1.40/M). Phase 2 continues online on the $K=4$ test split ($n=1,824$), where Flash rewards are judged by DeepSeek-R1 using the same v3 continuous rubric as all other arms.

All conditions use the simulation-tuned hyperparameters from Experiments 02–03: $\alpha=0.1$, $n_{\text{eff}}=10$, $\gamma=0.995$, consistent across all main-body experiments.

Two Phase 2 strategies are compared per budget tier:

- **Fixed Policy (uniform 1/4):** Equal allocation across all $K=4$ arms. The simplest possible onboarding strategy—no routing intelligence, no budget awareness.
- **BanditGPT (transfer):** Phase 1 posteriors carry over; Flash gets a T-shirt prior. BudgetPacer continues without reconfiguration. Three budget targets span the constraint regime: tight ($B=\$2.3\times 10^{-4}/\text{req}$), moderate ($B=\$6.6\times 10^{-4}/\text{req}$), and loose ($B=\$1.9\times 10^{-3}/\text{req}$). An unconstrained BanditGPT baseline (no pacer) serves as a quality ceiling. Results: 20 seeds (9000–9019).

A.2.3 Results (Figure ??).

Strategy	Budget	Reward	Cost (\$/req)	Flash %	Compliance?
Fixed Policy	any	0.893	\$4.05e-3	25.1%	No (6–17 $\times$)
BanditGPT	tight	0.853	\$2.34e-4	10.7%	✓
BanditGPT	moderate	0.889	\$6.60e-4	19.4%	✓
BanditGPT	loose	0.913	\$1.80e-3	28.1%	✓
BanditGPT	unconstrained	0.924	\$7.68e-3	20.8%	—

Flash is discovered in every seed. Across all 80 BanditGPT trials (20 seeds $\times$ 4 budget tiers), Flash is first selected within ~ 340 post-onboarding steps and reaches sustained adoption (windowed share $> 10\%$ for 50 consecutive steps) by step ~ 370 . Under the unconstrained ceiling, Flash stabilises at $21.4 \pm 1.4\%$ of traffic—comparable to its uniform-1/4 share but *contextually allocated* (the bandit routes Flash to prompts where it matches Gemini-Pro quality at $10\times$ lower cost).

Budget determines adoption speed, not whether adoption occurs. At loose budgets, Flash reaches 33% of traffic in the final 200 steps, comparable to the unconstrained ceiling (21%). At tight budgets, the pacer limits Flash to 10%, just above the sustained-adoption threshold—the router correctly identifies that Flash (blended $\$1.40/\text{M}$) is cheaper than Mistral ($\$1.00/\text{M}$) but more expensive than Llama ($\$0.10/\text{M}$), and allocates accordingly.

Flash displaces the right arm. Displacement analysis (early 200 vs. late 200 steps of Phase 2) reveals a clear pattern:

- **Under budget pressure** (tight and moderate): Flash takes share primarily from **Mistral-Large** (-15.1 pp at tight, -15.5 pp at moderate). This is economically rational: Flash offers Mistral-level quality (~ 0.92) at a lower blended cost ($\$1.40$ vs. $\$1.00/\text{M}$).
- **Unconstrained:** Flash takes share from **Gemini-Pro** (-20.4 pp). Without cost pressure, the bandit uses Flash as a near-Pro-quality alternative at $10\times$ lower cost.

The BudgetPacer requires no reconfiguration. Figure ??(c) shows cost compliance throughout the trajectory. BanditGPT meets each budget target to within 1% across both phases, despite the portfolio expanding from $K=3$ to $K=4$ mid-trajectory. The dual variable λ_t adjusts smoothly: when Flash's lower cost reduces the running average spend, λ_t relaxes, freeing budget for contextual exploration.

Fixed Policy: budget-oblivious baseline. The uniform-1/4 baseline highlights the cost of ignoring budget constraints. By giving Gemini-Pro 25% of traffic, it spends $\$4.05e-3$ per request—exceeding the tight budget by $17\times$ and even the loose budget by $2\times$. It achieves a reward of 0.893, which BanditGPT matches or exceeds at moderate and loose budgets (0.889 and 0.913) while spending $6\times$ and $2\times$ less.

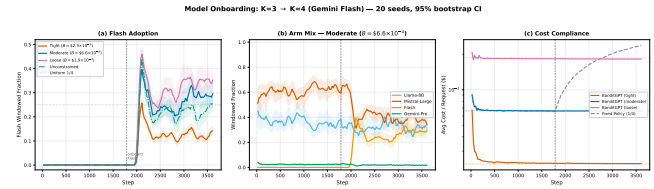


Figure 8:

Model onboarding under budget constraints ($K=3 \rightarrow K=4$). A $K=3$ router (Llama-8B, Mistral-Large, Gemini-Pro) learns on 1,785 prompts with warmup priors, then onboards Gemini-2.5-Flash as a fourth arm via `register_model()` (dashed vertical line). Phase 2 continues online on the $K=4$ test split (1,824 prompts). (a) Flash adoption trajectory: the windowed selection fraction of Flash rises from zero after onboarding and stabilises within ~ 500 steps across all budget tiers. Loose and unconstrained budgets reach 30–35% Flash share; tight budgets plateau at $\sim 10\%$. The grey dotted line marks 25% (uniform 1/4 baseline). (b) Full arm composition under the moderate budget: Flash primarily displaces Mistral-Large (-15.5 pp), taking over the mid-tier quality niche at lower cost. (c) Cost compliance on log scale: BanditGPT (solid) tracks each budget target (dotted) within $\sim 1\%$ across the full trajectory, while the Fixed Policy (dashed grey) exceeds all targets by 6–17 $\times$. 95% bootstrap CI across 20 seeds.

A.2.4 Practical Implications.

One API call, zero downtime. At moderate and loose budgets—the regime of most production deployments—onboarding a new model into BanditGPT requires a single call to `register_model()` with the model's pricing. No offline evaluation, no policy retraining, no budget reconfiguration. The bandit discovers the model's quality–cost niche from live traffic and the pacer auto-adjusts.

Budget-quality Pareto improvement. Flash is a Pareto-improving arm: it offers near-Pro quality at Mistral-level cost. A static router would require human analysis to discover this; BanditGPT identifies and exploits it automatically, shifting the portfolio's cost–quality frontier.

Tight budgets require deliberate exploration. Under tight budget constraints, the pacer's exploration budget is limited, slowing (but not preventing) Flash adoption. Practitioners running at very tight margins who want to onboard an expensive newcomer should temporarily increase the budget target to allow initial exploration, then tighten once the bandit calibrates.